1 Install Julia

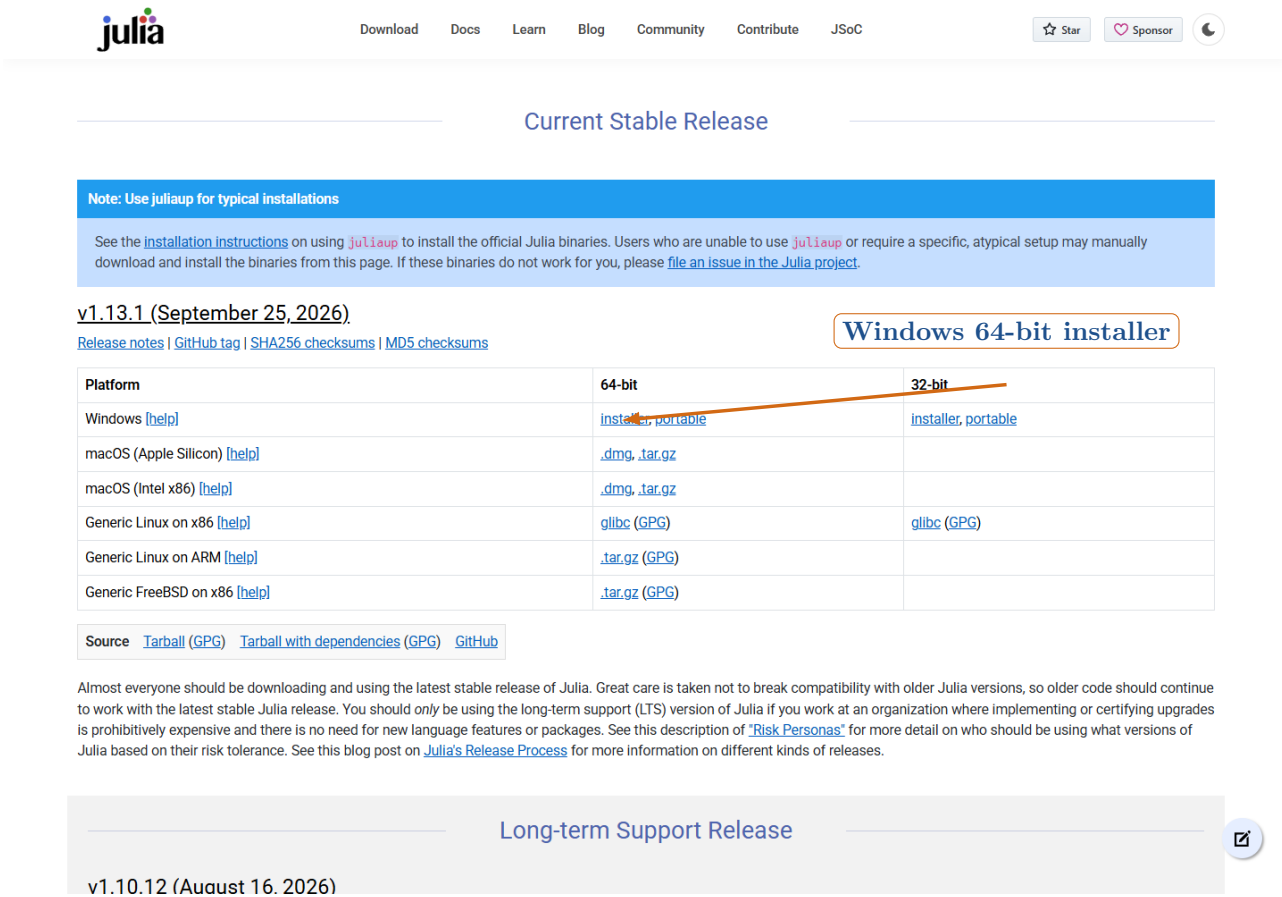
Start here. One installation prepares JFEM, FGen, PANDEATOR and the results viewer.

1. Open the official Julia downloads page.
2. Choose the installer for your computer. This checkout uses the **Julia 1.13 release line**, tested with **1.13.1**.
3. Run the installer. On Windows, enable **Add Julia to PATH** if offered. Close the installer, then open a new terminal.
4. Check that Julia is available:

```
julia --version
```

With Juliaup, select a compatible runtime using `juliaup add 1.13`, then `julia +1.13 --version`. A standalone installation uses its executable directly.

Ready when: the terminal reports Julia 1.13.x.



Download Julia from its official site. The page appearance and latest release can change; use the release line required by this checkout.

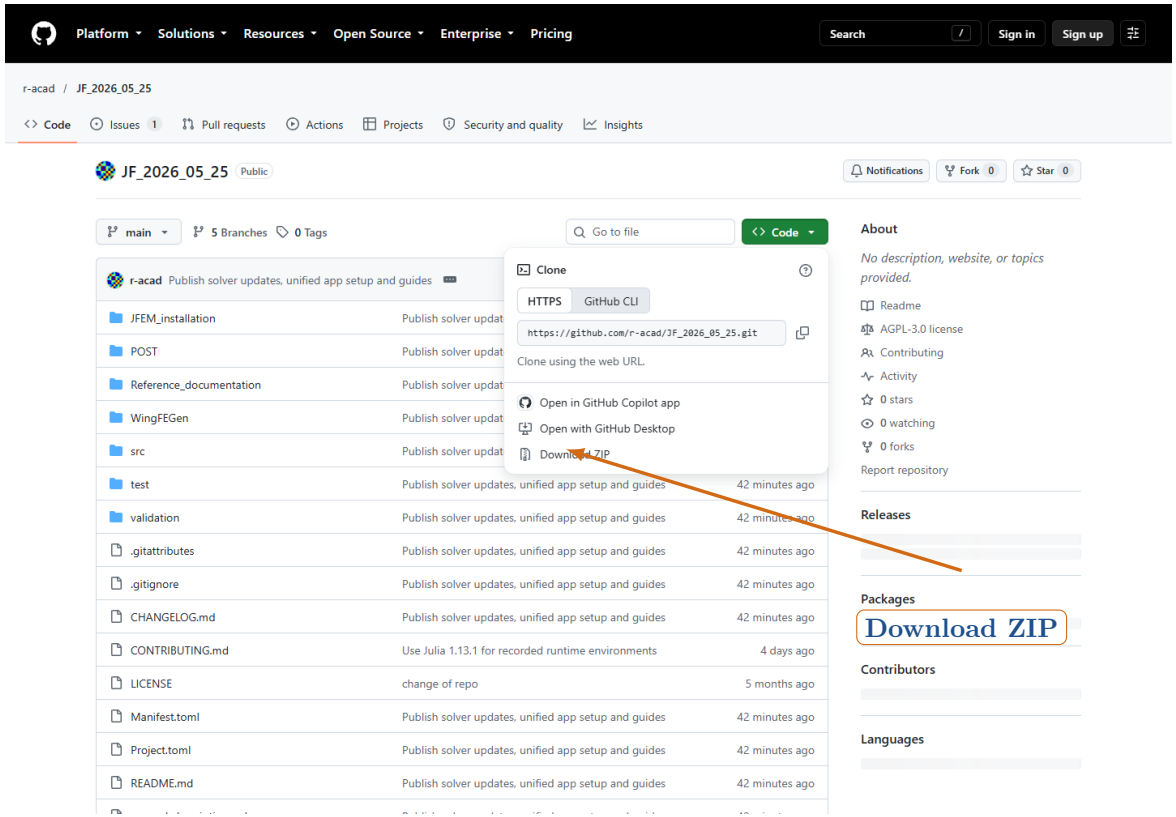
2 Download and extract JFEM

Keep the complete repository together in a writable folder.

1. Open github.com/r-acad/JF_2026_05_25.
2. Click **Code**, then **Download ZIP**.
3. Extract the whole ZIP. On Windows, right-click it and choose **Extract All**. On macOS/Linux, use your archive tool.
4. Open the extracted folder. It must contain `setup.cmd`, `setup.sh`, `WingFEGen` and `POST`.

Use a folder such as `C:\Work\JFEM` or `~/JFEM`. Its exact name is unimportant.

Ready when: you are inside the extracted folder, not browsing inside the ZIP.



Code → Download ZIP → Extract All → open the extracted folder.

Keep `src`, `JFEM_installation`, `WingFEGen`, `POST` and the root `Project.toml`/`Manifest.toml`. The applications share files.

3 Run setup once

Install packages and check all four applications. Internet is needed for missing downloads.

1. On Windows, open the extracted folder in Explorer. Click its address bar, type **powershell**, and press Enter. In the new terminal, run:

```
.\setup.cmd
```

On macOS/Linux, open a terminal in that folder:

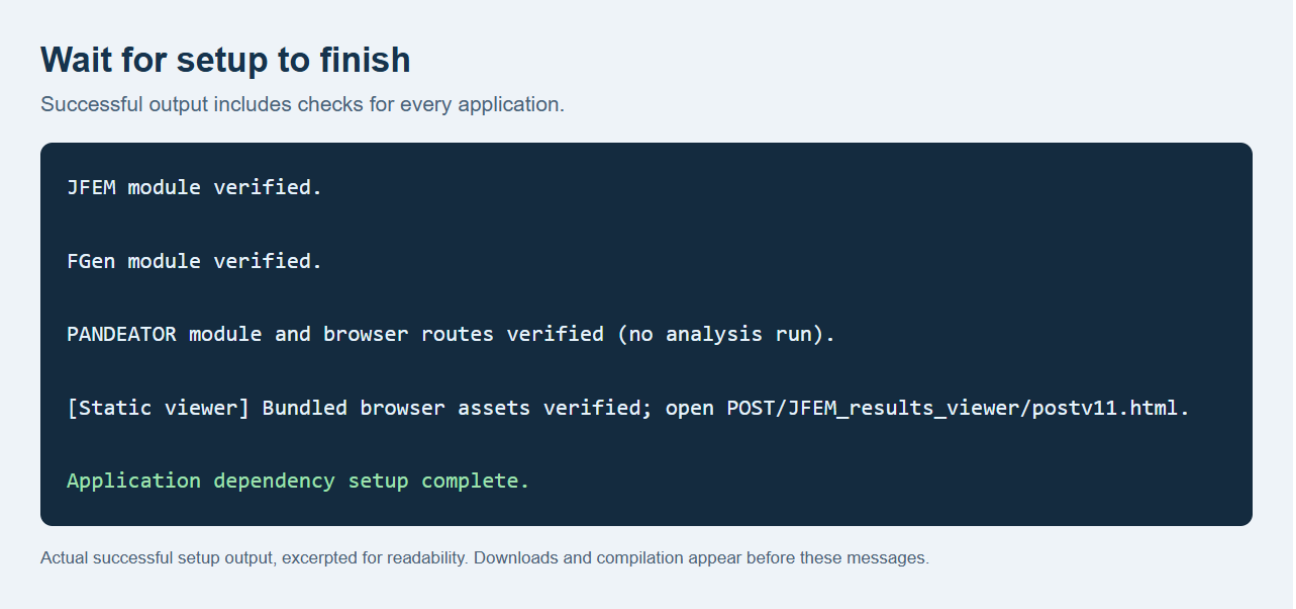
```
bash setup.sh
```

2. Leave the window open. Downloads and first compilation can take several minutes.

3. Wait for the completion summary. Resolve any reported error before continuing.

This prepares two Julia environments and checks JFEM, FGen, PANDEATOR and the static viewer. Normal setup does not build a sysimage or run your analysis.

Ready when: all selected application checks pass.



Successful setup log excerpt, typeset for this guide. Your paths and elapsed times will differ.

4 Open FGen

Build a lifting-surface FEM, or read an existing Reference deck.

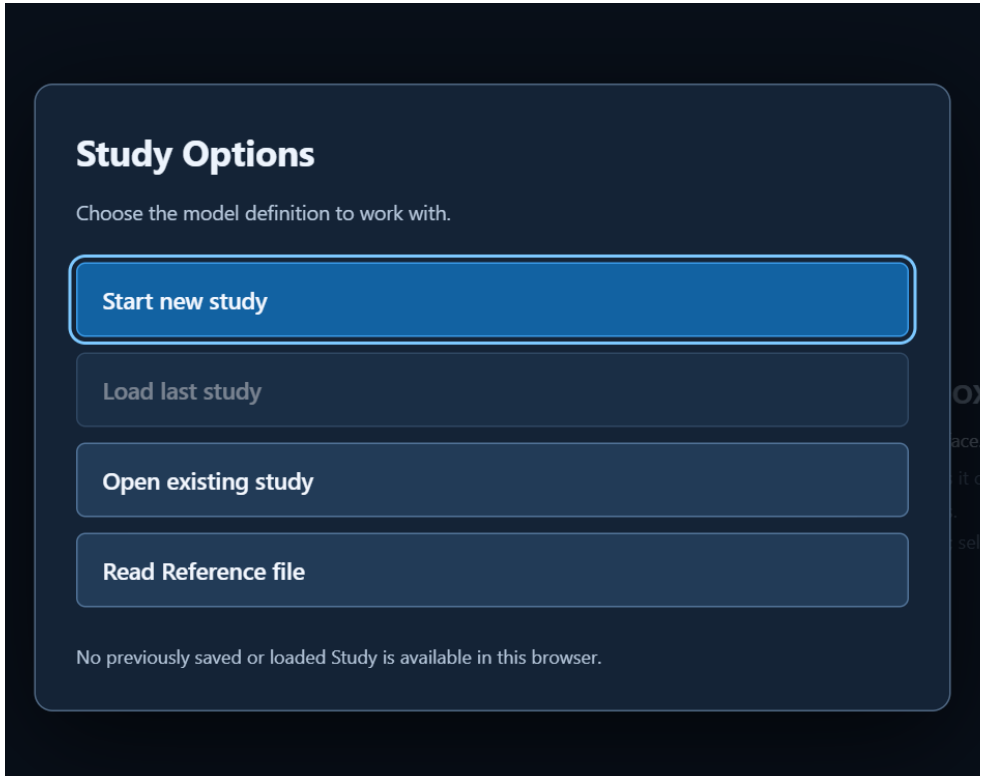
1. On Windows, open **WingFEGen** and double-click **run_fe_generator.cmd**.

2. Wait for the local browser page. Its usual address is `http://127.0.0.1:8080`.

3. Choose **Start new study**. To import a deck instead, choose **Read Reference file**.

4. Save your work using **Project**. Keep the server terminal open while using FGen.

Ready when: the Study Options dialog appears.
The separate FGen user guide explains geometry, loads, analyses, results and saved studies.



The startup dialog in a fresh FGen session.

Portable launch command (all platforms), from the extracted root:

```
julia --startup-file=no --project=WingFEGen WingFEGen/run.jl
```

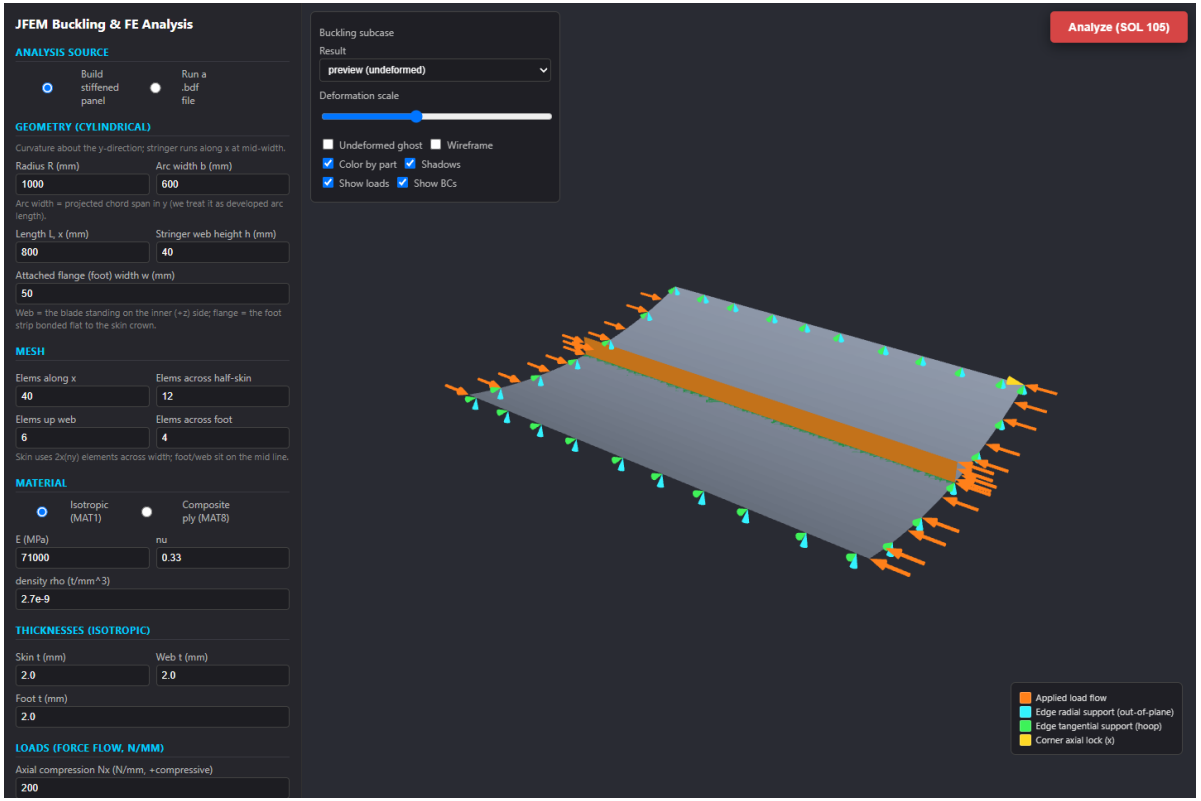
To use another port and open the browser yourself, append `--port 9000 --no-browser`. Use the URL printed by the server.

5 Open PANDEATOR

Generate and analyse a stiffened panel. This is a separate browser application.

1. Open **POST**, then **PANDEATOR_APP**.
2. On Windows, double-click **RUN_PANDEATOR_WINDOWS.cmd**.
3. Wait while the server starts. Its usual address is `http://127.0.0.1:8088/`.
4. Use the panel form and preview. Keep the terminal open while working.

Ready when: the panel form opens in your browser. Startup can warm solver methods before the server is ready. The first analysis may still compile additional methods.



PANDEATOR uses the same root solver environment prepared in step 3.

On macOS/Linux, from the extracted root:

```
bash POST/PANDEATOR_APP/RUN_PANDEATOR_MAC_LINUX.sh
```

Both PANDEATOR launchers accept `--port 8090 --no-open`. Stop your server with Ctrl+C after saving its outputs.

6 Run a small model and view its results

The standalone solver and the static viewer also work without either app server.

1. Run the bundled tiny SOL101 example.

In PowerShell, from the extracted root:

```
$examples = "JFEM_installation\examples\precompile"
$deck = "$examples\sol101_quad_static.bdf"
.\jfem.cmd "$deck" output\first_run
```

In macOS/Linux:

```
examples=JFEM_installation/examples/precompile
deck="$examples/sol101_quad_static.bdf"
bash jfem "$deck" output/first_run
```

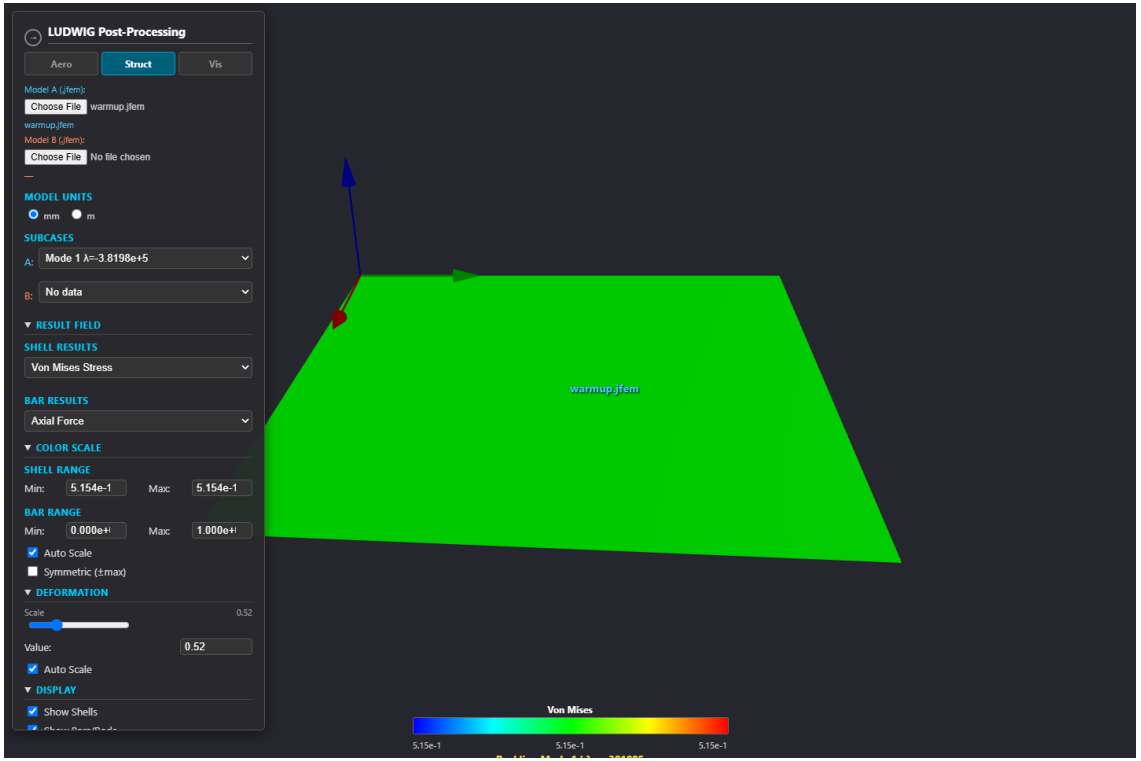
2. Open the viewer file:

`POST/JFEM_results_viewer/postv11.html`

3. Select Struct, then Model A (.jfem).

Use its **Choose File** control to load the generated `.jfem` file in `output/first_run`.

Ready when: the solver exits successfully and the viewer reads its result file.



Tiny demonstration output shown; this is not a physical benchmark. The viewer reads existing results and does not launch analyses. Keep the full `POST` tree for its local assets.

The default export includes `.jfem`, report and JSON files. Adding `-rs` before the input deck exports only report/JSON, so it does *not* provide the binary file needed by this viewer.

7 Optional: build a local sysimage

Use this after ordinary setup works, to reduce repeated startup and first-use compilation.

1. Stop your own solver/server processes before replacing an image they may have loaded.

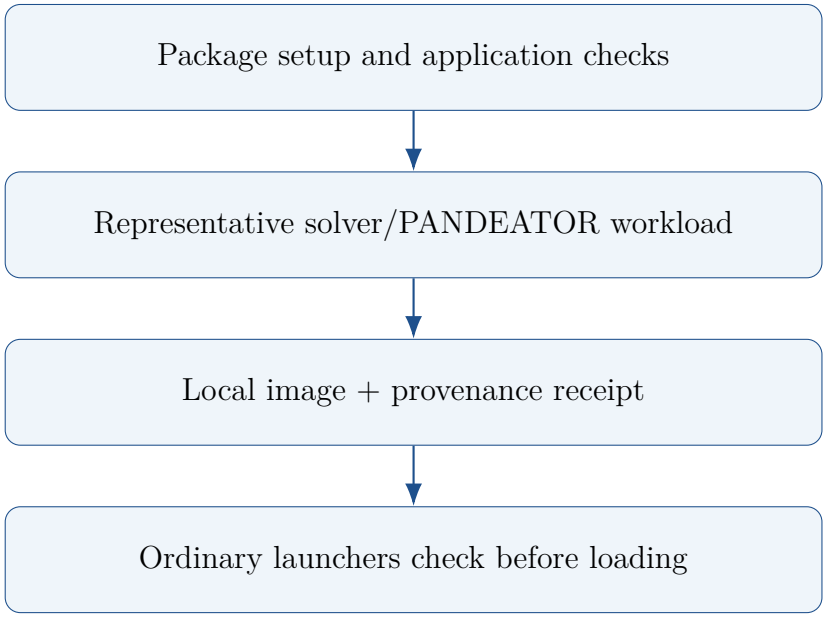
2. From the extracted root, run:

```
# Windows
.\setup.cmd --sysimage
# macOS/Linux
bash setup.sh --sysimage
```

3. Leave the terminal open until the build finishes. It runs small representative analyses and can use substantial time, CPU and memory.

4. Keep the new image *and* its adjacent `.jfem.toml` receipt. Then use the ordinary launch commands from the preceding steps.

Ready when: the image and receipt exist and a small ordinary run succeeds.



Standard file: `sysimage/OpenJFEM_sysimage.dll` on Windows, `.so` on Linux, `.dylib` on macOS.
PackageCompiler needs a C compiler: provided automatically on Windows when needed; use an appropriate clang/gcc toolchain on macOS/Linux.

This image covers OpenJFEM and PANDEATOR. Eligible native solver runs launched by FGen can use it; the separate FGen server is not compiled into it. New code paths may still compile. The static viewer uses no Julia image.

A If you need more control

Quick repairs and sysimage reference. The normal numbered process remains the starting point.

Common fixes

- **Julia not found:** reopen the terminal; check `julia --version`. Setup accepts `JFEM_JULIA`; FGen also accepts `WINGFEGEN_JULIA`. Use an executable path, without flags.
- **Downloads interrupted:** fix the network/proxy problem and rerun setup. A prepared matching cache is required for `--offline`.
- **Browser cannot connect:** wait for readiness, keep Julia running, and use the printed URL/port.
- **After an update:** retain studies/results, update the complete checkout, and rerun setup.

Verify an image before deliberate direct use

```
julia --startup-file=no --project=.
    JFEM_installation/julia_tools/check_sysimage.jl .
    sysimage/OpenJFEM_sysimage.dll
```

Use `.so`/`.dylib` on Linux/macOS. Success is silent with exit code zero. Automatic launchers fall back to ordinary package loading if the receipt no longer matches.

Rebuild after runtime, code/dependency, checkout-location, machine/CPU or representative-workload changes. Repeat the platform command on page 7 with `--sysimage`. Do not force an old image with Julia's `-J` option.

Select a workload or build thread count

```
.\setup.cmd --sysimage --deck path/to/representative.bdf --sysimage-threads=4
```

Repeat `--deck`, or use `--manifest cases.json` (a JSON `cases` array). `--flags NAME=value,OTHER=value` adjusts workload settings. These options require `--sysimage`; `--dry-run` prints the plan. Build thread count is not a memory cap.

Existing CLICK/RUN scripts

The existing platform wrappers in `JFEM_installation` delegate to setup with `--sysimage`:

`CLICK_WINDOWS_INSTALL_PACKAGES_AND_CREATE_SYSIMAGE.cmd`

`CLICK_MAC_INSTALL_PACKAGES_AND_CREATE_SYSIMAGE.command`

`RUN_LINUX_INSTALL_PACKAGES_AND_CREATE_SYSIMAGE.sh`

Original direct builder (after setup)

```
julia --threads=auto --startup-file=no --project=.
    JFEM_installation/julia_tools/deploy_fast.jl
    --sysimage=sysimage/OpenJFEM_sysimage.dll
```

A custom `--sysimage=PATH` is not auto-discovered. Check its receipt before using `-J`; keep it with the root solver project. Direct builder use is not all-app setup.

Diagnose without an image

```
julia --startup-file=no --project=. JFEM_installation/julia_tools/run_bdf.jl
    JFEM_installation/examples/precompile/sol101_quad_static.bdf
    output/source_check
```

Further help: `JFEM_installation/INSTALLATION.md`; Julia installation; PackageCompiler sysimages. FGen's illustrated user guide is under `Reference_documentation/00_Getting_Started/`.